

Bereiche

1. Firmware-Entwicklung

- Treiber für externe Hardware (Sensor-Readings, Steuerung)
- Kommunikationsprotokolle (I²C, SPI, UART)
- Low-Level-Logik (Interrupts, Timers, ADC/DAC)
- Datenvorverarbeitung (Filterung, Kalibrierung)

2. **Benutzeroberfläche (UI)**

- Touchscreen-Steuerung (Buttons, Slider, Menüs)
- Anzeige von Echtzeitdaten (Graphen, Statusindikatoren)
- Konfigurationsmenüs (Einstellungen speichern/laden)

3. **Kommunikationsschnittstellen**

- WiFi/Bluetooth (MQTT, HTTP, WebSocket)
- APIs für externe Systeme (REST, Webhooks)

4. **Datenverarbeitung & Logging**

- Datenspeicherung (SD-Karte, Flash)
- Fehlerprotokollierung (Crash Reports, Debug-Logs)

5. **Power-Management**

- Batterieüberwachung (Ladezustand, Sleep-Modi)

6. **Sicherheit**

- Secure Boot/Firmware-Signatur
- OTA-Updates (sicherer Firmware-Update-Prozess)

7. **Testing & Validation**

- Unit-Tests (Hardware-unabhängige Logik)
- Integrationstests (Gesamtsystem unter Last)
- Benutzertests (Usability, Fehlertoleranz)

8. Dokumentation

- Schaltpläne, Pinbelegungen
- Code-Kommentare, Architekturdokumentation
- Benutzerhandbuch (Bedienungsanleitung)

9. Deployment & Wartung

- Feldtests (Pilotinstallationen)
- Update-Strategie (OTA, manuelle Patches)

Kernlogik & Datenverarbeitung

Ziel: Funktionsfähige Logik ohne UI, aber mit Hardware-Anbindung.

1. Datenpipeline

- Zustandsautomaten

2. Speicherung & Logging

- SD-Karte/Flash für Datenspeicherung implementieren.
- Fehlerspeicher (z. B. letzter bekannter Sensorwert bei Absturz).

3. Testframework

- Automatisierte Tests für kritische Logik (z. B. `assert(getTemperature() != NaN)`).

Phase 3: UI-Entwicklung

Ziel: UI aufbauen, die auf bereits getesteter Hardware/Logik aufsetzt.

1. UI-Prototyping

- Mockups mit [M5Stack UI-Library](#) erstellen.
- Navigation zwischen Screens (z. B. Hauptmenü → Einstellungen).

2. Echtzeit-Anbindung

- UI-Elemente mit Hardware-Logik verknüpfen (z. B. Live-Graph aus Sensordaten).
- Touch-Inputs in Steuerbefehle umsetzen.

Phase 4: Kommunikation

Ziel: Anbindung an externe Systeme, erst nachdem lokal alles stabil läuft.

1. WiFi

- Verbindungsmanagement (Reconnect-Logik, Failover).

2. Sichere Kommunikation

- API-Key-Management (nicht hartcodieren!).

3. Cloud-Logik

- OTA-Update-Funktion testen.

Phase 5: Finale Integration & Optimierung

1. Power-Management

- Stromverbrauch messen (z. B. mit Power Profiler).

2. Sicherheitsaudit

- Firmware signieren.
- Sensible Daten (WiFi-Passwörter) verschlüsselt speichern.

3. End-to-End-Tests

- 24h-Dauerlauf unter Last (z. B. Speicherlecks prüfen).
- Benutzerakzeptanztest (Usability-Feedback einholen).

2.1 FIRMWARE

Kernkomponenten der Firmware-Logik

1. Initialisierung & Hardware-Setup

- GPIO-Pins, Kommunikationsprotokolle (I²C/SPI/UART) konfigurieren.
- Sensoren/Aktoren kalibrieren.

2. Sensor/Aktor-Treiber

- Bibliotheken für externe Sensoren (z. B. BME280, MPU6050) einbinden.
- Wrapper-Funktionen für Hardware-Interaktion schreiben (z. B. readSensor()).

3. Zustandsautomaten & Logik

- Endliche Zustandsmaschinen (FSM) für Betriebsmodi (z. B. "Normal", "Sleep", "Error").
- Zeitkritische Logik mit **Interrupts** oder **FreeRTOS-Tasks** (bei ESP32-S3).

4. Datenverarbeitung

- Filteralgorithmen (Moving Average, Medianfilter).
- Kalibrierung (z. B. lineare Skalierung von Rohwerten zu physikalischen Einheiten).

5. Fehlerbehandlung

- Timeouts bei Sensorabfragen (z. B. 500ms Wartezeit → Fehlercode).
- Watchdog-Timer zur Überwachung von Hängern.

6. Nicht-blockierender Code

- **Vermeide delay()** – nutze stattdessen millis() für zeitgesteuerte Aktionen.
- Zustandsmaschinen für sequenzielle Abläufe (z. B. "Warte auf Sensorantwort → Verarbeite Daten → Sende Ergebnis").